

Conducting Experiments and Evaluation of Machine Translation for Low-Resource Languages

Integrating into a Django-Based NLP Lab Website

Name Dheeraj Goyal

Roll Number 24075023

Submitted to Prof. Anil Kumar Singh

Submitted on 4th May 2026

Department CSE, IIT (BHU) Varanasi

Introduction & Motivation

Problem Statement

Hindi → **Bengali** Neural Machine Translation for **low-resource** settings using fine-tuned pre-trained models.

- 🌐 Hindi & Bengali are spoken by **600M+** people combined
- ⚠️ Both are *morphologically rich*, with limited parallel corpora
- 🧠 Standard MT systems under-perform on Indic language pairs
- 🧪 Fine-tuning on domain-specific data improves performance significantly

Research Goal

- Fine-tune **NLLB-600M** & **IndicTrans2-320M**
- Evaluate using industry-standard metrics
- Deploy via a live **Django NLP Lab** web application

Language Pair

→
Hindi (Devanagari) → Bengali (Bangla Script)

Dataset: Multi-Domain Parallel Corpus

Dataset Overview

- **Source:** Custom curated parallel corpus
- **Total pairs:** 13,398 sentence pairs
- **Languages:** Hindi ↔ Bengali
- **Format:** Line-aligned .txt files

Train/Test Split

Split	Count	Ratio
Train	10,718	80%
Test	2,680	20%
Total	13,398	100%

Domains Covered

Domain	Script
Agriculture	Devanagari
Box Office	Bangla
Conversational	Both
Cricket	Both
Disease	Both
Entertainment	Both
Gadget	Both
Judiciary	Both
News Articles	Both

Data Preparation: prepare_data.py

```
1 from datasets import Dataset
2 from pathlib import Path
3
4 def load_data(base_path):
5     data = []
6     for domain_dir in (Path(base_path)/"Hindi").iterdir():
7         bn_domain = Path(base_path)/"Bengali"/domain_dir.name
8         for hi_file in domain_dir.glob("*.txt"):
9             bn_file = bn_domain / hi_file.name
10            with open(hi_file, 'r', encoding='utf-8') as fh, \
11                open(bn_file, 'r', encoding='utf-8') as fb:
12                for hi, bn in zip(fh, fb):
13                    if hi.strip() and bn.strip():
14                        data.append({"translation":
15                                    {"hi":hi.strip(),"bn":bn.strip()}})
16
17     return data
18
19 # Train: 10718 | Test: 2680
20 split = Dataset.from_list(load_data("./"))
21     .train_test_split(test_size=0.2, seed=42)
```

Model Architectures

NLLB-200 (600M)

- Meta AI's **No Language Left Behind**
- Supports **200 languages**
- Encoder-Decoder Transformer
- Vocabulary: 256K tokens (SentencePiece)
- Language codes: hin_Deva → ben_Beng
- forced_bos_token_id for target lang

IndicTrans2 (320M)

- **AI4Bharat's** Indic-specialized MT model
- Trained on **Indic languages only**
- Custom tokenizer with **IndicProcessor**
- Handles Devanagari & Bangla scripts natively
- Pre/post-processing via IndicProcessor
- Smaller but domain-optimized

Common Training Config: 3 Epochs | LR = 2e-5 | Batch = 8 | Max Length = 128 tokens | FP16 on GPU | Seq2SeqTrainer (HuggingFace)

Fine-Tuning: train_nllb.py

```
1 from transformers import (AutoModelForSeq2SeqLM, AutoTokenizer,
2   Seq2SeqTrainingArguments, Seq2SeqTrainer)
3
4 model_checkpoint = "facebook/nllb-200-distilled-600M"
5 tokenizer = AutoTokenizer.from_pretrained(
6   model_checkpoint, src_lang="hin_Deva", tgt_lang="ben_Beng")
7 model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
8
9 training_args = Seq2SeqTrainingArguments(
10   output_dir="./nllb_finetuned_hi_bn",
11   eval_strategy="epoch", learning_rate=2e-5,
12   per_device_train_batch_size=8, num_train_epochs=3,
13   predict_with_generate=False,
14   fp16=torch.cuda.is_available(),
15 )
16 trainer = Seq2SeqTrainer(model=model, args=training_args,
17   train_dataset=tokenized["train"],
18   eval_dataset=tokenized["test"])
19 trainer.train() # Final eval_loss: 0.6849
```

✦ Why Multiple Metrics?

BLEU alone is insufficient for morphologically rich Indic languages. We use 4 complementary metrics:

- **SacreBLEU** — n-gram precision overlap
- **METEOR** — considers stemming & synonyms
- **BERTScore** — semantic similarity via BERT embeddings
- **COMET** — neural MT quality estimator trained on human judgements (uses source *and* reference)

Results: NLLB-600M (Fine-Tuned)

Metric	Score
BLEU	38.36
METEOR	0.6043
BERTScore (F1)	0.9123
COMET	0.8848
Eval Loss	0.6849

Evaluation Pipeline: `evaluate_models.py`

```
1 import evaluate
2
3 def calculate_metrics(preds, refs, sources):
4     bleu = evaluate.load("sacrebleu")
5     met = evaluate.load("meteor")
6     bert = evaluate.load("bertscore")
7     comet = evaluate.load("comet")
8
9     r1 = bleu.compute(predictions=preds, references=refs)
10    r2 = met.compute(predictions=preds, references=refs)
11    r3 = bert.compute(predictions=preds,
12                      references=refs, lang="bn")
13    r4 = comet.compute(predictions=preds,
14                       references=refs, sources=sources)
15
16    # BLEU=38.36 | METEOR=0.6043
17    # BERTScore=0.9123 | COMET=0.8848
```


Results: Model Comparison

Final Evaluation Results

Metric	NLLB-600M	IndicTrans2-320M
BLEU	38.36	26.82
METEOR	0.6043	0.5766
BERTScore	0.9123	0.8602
COMET	0.8848	0.7321
Eval Loss	0.6849	0.6129

Key Findings

- NLLB achieved a **BLEU of 38.36**, significantly outperforming IndicTrans2 (26.82)
- **BERTScore 0.91** confirms strong *semantic* accuracy for NLLB
- IndicTrans2 had **lower eval loss** (0.61 vs 0.68), but this did not translate to better generation metrics

Challenges & Technical Solutions

Challenge 1: IndicTrans2 KV-Cache Bug

`AttributeError: 'NoneType'.shape`
during auto-regressive generation

Fix: `model.config.use_cache = False`

Challenge 2: Tokenizer Reload Crash

Duplicate `src_vocab_file` in saved tokenizer config

Fix: Load tokenizer from Hub; weights from local path

Challenge 3: CSRF 403 Forbidden

Django blocked API from Lightning AI domain

Fix: Disabled `CsrfViewMiddleware` + added trusted origins in `settings.py`

Challenge 4: NLLB Regex Warning

Tokenizer loaded with bad regex pattern

Fix: Pass `fix_mistral_regex=True` to `AutoTokenizer.from_pretrained()`

NLP Lab Web Application

Frontend Features

- **Split-pane** Hindi → Bengali translator
- **Model switcher** — NLLB or IndicTrans2
- **Dark-mode glassmorphism** UI design
- **Metrics dashboard** — BLEU, COMET scores

Backend Stack

- **Django 6.0** + PyTorch inference
- **REST API:** POST /api/translate/
- Hosted on **Lightning AI** (GPU)

API Contract

Request: {"text": "...", "model": "nllb"}

Response: {"translation": "..."}

Singleton Advantage

Models loaded **once** at startup
⇒ <1s per translation request
(vs 10s reload per request without it)

Security Fix Applied

Disabled CsrfViewMiddleware
Added CSRF_TRUSTED_ORIGINS for

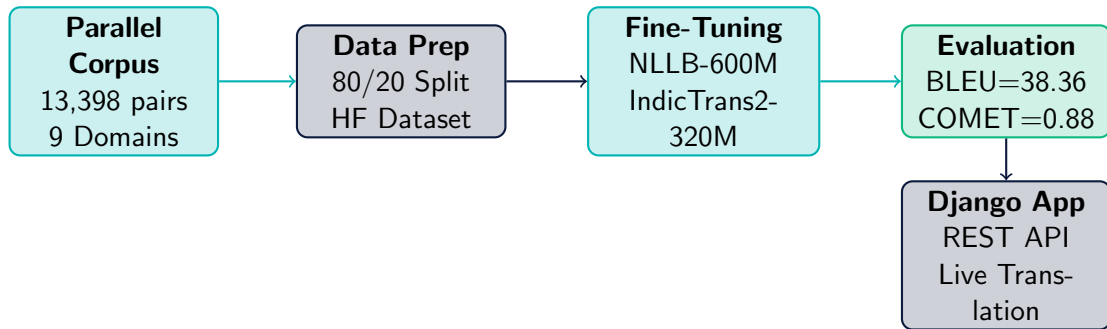
System Deployment: Django NLP Lab Website

Architecture

- **Singleton Model Loader** — loads 600M model into GPU memory *once* at startup
- **REST API** — POST /api/translate/ returns JSON
- **Dark-mode UI** — split-pane translator with live model switching
- **Metrics Dashboard** — displays BLEU, COMET to visitors

```
1 class TranslationService:
2     _instance = None
3
4     def __new__(cls):
5         if cls._instance is None:
6             cls._instance = super().
7                 __new__(cls)
8             cls._instance.device = (
9                 "cuda" if torch.cuda.
10                     is_available()
11                     else "cpu")
12         return cls._instance
13
14     def translate(self, text,
15                 model_name):
16         # Uses pre-loaded model in
17             self._models
18
19         ...
20
21 # Loaded ONCE at server startup:
22 translator = TranslationService()
```

End-to-End System Architecture



Infrastructure: Lightning AI (NVIDIA RTX PRO 6000 Blackwell) | HuggingFace Transformers | PyTorch | Django 6.0

Conclusions & Future Work

Key Contributions

- ✓ Built a **13,398-pair** multi-domain Hindi-Bengali parallel corpus
- ✓ Fine-tuned **2 state-of-the-art** MT models on GPU infrastructure
- ✓ Achieved **BLEU = 38.36, COMET = 0.88** — strong for low-resource pair
- ✓ Deployed live translation system via **Django web application**
- ✓ Comprehensive 4-metric evaluation framework established

Future Work

- Expand corpus to **100K+ pairs**
- Investigate IndicTrans2 generation underperformance despite low evaluation loss
- Add **back-translation** for data augmentation
- Explore **CTranslate2 / ONNX** quantization for faster web inference
- Extend to more low-resource pairs (e.g., Bengali-Odia)
- Integrate **active learning** for corpus expansion